Hierarchical k-GNN Explanations via Generative Interpretation Networks

Comparing 1-GNN and 1-2-GNN for Model-Level Graph Classification Interpretability

[Author Name]

March 2026

Abstract

Graph Neural Networks (GNNs) are powerful tools for graph classification, yet their decision processes remain opaque. Higher-order k-GNNs, which operate on k-element subsets of nodes, provably increase expressive power beyond the Weisfeiler–Leman hierarchy, but it is unclear whether this additional expressiveness leads the model to learn qualitatively different internal representations. We investigate this question by comparing a standard 1-GNN with a hierarchical 1-2-GNN using the GIN-Graph framework for model-level explanation generation. Our pipeline trains k-GNN classifiers on the MUTAG and PROTEINS benchmarks, then uses a Wasserstein GAN with gradient penalty to generate class-representative explanation graphs guided by the pretrained classifier. We introduce a fully differentiable dense wrapper that maintains gradient flow through both node features and adjacency matrices for hierarchical models. On MUTAG, both models achieve identical classification accuracy (89.5%), yet they generate strikingly different explanations: the 1-GNN associates mutagenicity with nitrogen-containing structures while the 1-2-GNN favours halogenated compounds, revealing that the two architectures encode different structural features of the same class. Cross-model classification of 500 generated graphs per class confirms this divergence quantitatively—on PROTEINS, 1-GNN-generated Non-Enzyme graphs are rejected by the 1-2-GNN (9.4% agreement), while 1-2-GNN-generated graphs are accepted by both models (98–100% agreement), indicating that the 1-2-GNN learns a more selective decision boundary in its pairwise feature space. These results demonstrate that different GNN architectures develop qualitatively different internal representations of graph classes, and that model-level explanations can reveal these differences even when classification accuracy is comparable.

Contents

1 Introduction

Graph Neural Networks (GNNs) have become the standard approach for learning on graph-structured data, achieving strong results on tasks ranging from molecular property prediction to social network analysis. Despite their effectiveness, GNNs suffer from the same interpretability challenges as other deep learning models: their predictions are difficult to explain in human-understandable terms.

The expressiveness of standard message-passing GNNs is bounded by the 1-dimensional Weisfeiler–Leman (1-WL) graph isomorphism test [Morris et al.(2019)]. Morris et al. [Morris et al.(2019)] proposed k-GNNs that operate on k-element subsets of nodes, achieving expressiveness equivalent to the k-WL test. In theory, higher-order k-GNNs can distinguish graph structures that standard GNNs cannot.

A natural question arises: *does this additional structural expressiveness lead the model to learn different internal representations of graph classes, and can we observe these differences through generated explanations?* If a model captures richer structural patterns, the explanations it produces—representative graphs that characterise each class—may reveal what structural features each architecture has learned to associate with each class.

We investigate this question using the GIN-Graph framework [Sun et al.(2025)], which generates model-level explanation graphs through a GAN-based approach guided by a pretrained classifier. Model-level explanations aim to answer the question "what does a typical graph of class c look like according to the model?" by generating new graphs that maximise the model's confidence for that class while remaining structurally realistic. This contrasts with instance-level methods that explain individual predictions by identifying important subgraphs.

Specifically, we compare:

- A **1-GNN**: standard node-level message passing, equivalent to 1-WL;
- A **1-2-GNN**: hierarchical architecture combining node-level (1-GNN) and pairwise (2-GNN) message passing, achieving expressiveness beyond 1-WL.

Our contributions are:

1. A fully differentiable dense wrapper enabling GIN-Graph training with hierarchical k-GNN classifiers, maintaining gradient flow through adjacency matrices at both the 1-GNN and 2-GNN levels.
2. A corrected validation metric that computes actual cosine similarity between generated and real graph embeddings, replacing the original proxy of using prediction probability.
3. A qualitative comparison showing that the two architectures learn fundamentally different internal representations of graph classes, revealed through divergent explanation structures and near-zero cross-model agreement.

2 Background

2.1 Graph Neural Networks

A graph $G = (V, E)$ consists of a node set V with $|V| = n$ and an edge set $E \subseteq V \times V$. Each node $v \in V$ carries a feature vector $\mathbf{x}_v \in \mathbb{R}^d$. We denote the adjacency matrix as $\mathbf{A} \in \{0,1\}^{n \times n}$ where $A_{uv} = 1$ if $(u, v) \in E$, and the feature matrix as $\mathbf{X} \in \mathbb{R}^{n \times d}$ where row v is $\mathbf{x}_v$.

GNNs learn node representations through iterative *message passing*. At each layer t, a node updates its representation by aggregating information from its neighbours:

$$\mathbf{h}_v^{(t)} = \mathrm{UPDATE}\big(\mathbf{h}_v^{(t-1)},\ \mathrm{AGGREGATE}\big(\{\!\{\mathbf{h}_u^{(t-1)} : u \in \mathcal{N}(v)\}\!\}\big)\big)\,, \tag{1}$$

where $\mathcal{N}(v) = \{u \in V : (u, v) \in E\}$ denotes the neighbourhood of v, $\mathbf{h}_v^{(0)} = \mathbf{x}_v$ is the initial node feature, and $\{\!\{\cdot\}\!\}$ denotes a multiset.

The choice of UPDATE and AGGREGATE functions determines the specific GNN variant. In our implementation, these are parameterised by separate linear transformations for the self-feature and the aggregated neighbour features. After T layers of message passing, a *readout* function pools all node embeddings into a single graph-level representation:

$$\mathbf{h}_G = \mathrm{READOUT}\big(\{\mathbf{h}_v^{(T)} : v \in V\}\big)\,. \tag{2}$$

Common choices for READOUT include sum, mean, and max pooling over the node set. We use sum pooling throughout this work, as it preserves information about both the features and the number of nodes.

2.2 The k-WL Hierarchy and k-GNNs

The *Weisfeiler–Leman* (WL) graph isomorphism test is a classical algorithm that iteratively refines node colour labels based on the multiset of neighbour colours. Two graphs are distinguished if, after some number of iterations, they produce different multisets of colours. The 1-WL test operates on individual nodes.

A fundamental result in GNN theory is that standard message-passing GNNs are *at most* as powerful as the 1-WL test [Morris et al.(2019)]: if 1-WL cannot distinguish two graphs, no message-passing GNN can either. This means there exist structurally different graphs (e.g., certain regular graphs) that no standard GNN can tell apart.

The k-WL test generalises this by operating on k-tuples of nodes, achieving strictly increasing discriminative power: for all $k \geq 2$, $(k+1)$-WL is strictly more powerful than k-WL [Cai et al.(1992)]. Morris et al. [Morris et al.(2019)] proposed k-GNNs that achieve this higher expressiveness by performing message passing on k-element subsets of nodes ("k-sets") rather than individual nodes.

For a k-set $s = \{v_1, \ldots, v_k\} \subseteq V$, the *local neighbourhood* is:

$$\mathcal{N}_L(s) = \{s' \subseteq V : |s'| = k,\ |s \triangle s'| = 2,\ \text{the differing elements are adjacent in } G\}\,, \tag{3}$$

where $s \triangle s'$ denotes the symmetric difference. In words: a neighbour of s is obtained by replacing exactly one element of s with a new node that is adjacent (in the original graph G) to the removed node.

Example. Consider a 2-set $s = \{u, v\}$. Its neighbours are all pairs $\{v, w\}$ where $(u, w) \in E$ (replacing u with adjacent w) and all pairs $\{u, w\}$ where $(v, w) \in E$ (replacing v with adjacent w). This means the 2-GNN can capture pairwise relationships and bond-level patterns that are invisible to a 1-GNN operating on individual nodes.

2.3 GIN-Graph

GIN-Graph [Sun et al.(2025)] is a model-level explanation method that generates class-representative graphs using a generative adversarial approach. The key idea is to train a generator $\mathcal{G}$ to produce graphs that satisfy two objectives simultaneously:

1. **Realism**: generated graphs should be structurally similar to real graphs from the dataset, enforced by a WGAN-GP discriminator.
2. **Class specificity**: generated graphs should be confidently classified as the target class by the pretrained GNN, enforced by a classification guidance loss.

The generator maps noise $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ to a graph $(\tilde{A}, \tilde{X})$ using Gumbel-Softmax for differentiable discrete sampling. The training objective balances the two losses through a dynamic weighting schedule that shifts focus from realism (early training) to class specificity (late training).

3 Method

This section details the mathematical formulations of our two classifier architectures (1-GNN and 1-2-GNN), the GIN-Graph generator, the training procedure, and the differentiable dense wrapper that bridges them.

3.1 1-GNN Architecture

Our 1-GNN implements message passing with separate transformations for self-features and neighbour-aggregated features. At layer t, the update rule for node v is:

$$\mathbf{h}_v^{(t)} = \sigma\left(\mathbf{h}_v^{(t-1)}\mathbf{W}_1^{(t)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(t-1)}\mathbf{W}_2^{(t)}\right), \tag{4}$$

where $\mathbf{W}_1^{(t)}, \mathbf{W}_2^{(t)} \in \mathbb{R}^{d_{t-1} \times d_t}$ are learnable weight matrices, σ is the ReLU activation function $\sigma(x) = \max(0, x)$, and $d_0 = d$ (the input feature dimension). The self-transformation $\mathbf{h}_v^{(t-1)}\mathbf{W}_1^{(t)}$ allows the model to retain and transform the node's own representation, while the neighbour term $\sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(t-1)}\mathbf{W}_2^{(t)}$ aggregates structural context.

This can equivalently be written in matrix form for the entire graph:

$$\mathbf{H}^{(t)} = \sigma\Big(\mathbf{H}^{(t-1)}\mathbf{W}_1^{(t)} + \mathbf{A}\,\mathbf{H}^{(t-1)}\mathbf{W}_2^{(t)}\Big), \tag{5}$$

where $\mathbf{H}^{(t)} \in \mathbb{R}^{n \times d_t}$ is the matrix of all node representations at layer t, and $\mathbf{A} \in \{0, 1\}^{n \times n}$ is the adjacency matrix. The matrix product $\mathbf{A}\,\mathbf{H}^{(t-1)}$ computes the sum of neighbour features for every node simultaneously.

After $T = 3$ layers with hidden dimension $d_1 = d_2 = d_3 = 64$, the graph embedding is obtained via sum pooling:

$$\mathbf{h}_G^{(1)} = \sum_{v \in V} \mathbf{h}_v^{(T)} \in \mathbb{R}^{d_T}. \tag{6}$$

This embedding is fed through a two-layer classifier MLP with ReLU activation and dropout:

$$\hat{y} = \mathrm{softmax}\Big(\mathbf{W}_o\, \sigma(\mathbf{W}_c\, \mathbf{h}_G^{(1)})\Big), \tag{7}$$

where $\mathbf{W}_c \in \mathbb{R}^{d_T \times d_T}$ and $\mathbf{W}_o \in \mathbb{R}^{d_T \times C}$ with C being the number of classes.

3.2 2-GNN and the Hierarchical 1-2-GNN

The 2-GNN operates on 2-sets (unordered node pairs) rather than individual nodes. For each pair $s = \{u, v\}$ with $u < v$ (canonical ordering), the initial feature vector incorporates the 1-GNN embeddings of both nodes and an *isomorphism type* indicator:

$$\mathbf{f}_s^{(0)} = \big[\mathbf{h}_u^{(T)} \parallel \mathbf{h}_v^{(T)} \parallel \mathbb{1}[(u, v) \in E]\big] \in \mathbb{R}^{2d_T+1}, \tag{8}$$

where $\parallel$ denotes vector concatenation. The isomorphism type $\mathbb{1}[(u, v) \in E] \in \{0, 1\}$ indicates whether u and v are connected by an edge. This is crucial because it allows the 2-GNN to distinguish between pairs of connected nodes and pairs of unconnected nodes—a distinction that carries structural meaning (e.g., in a molecule, bonded vs. non-bonded atom pairs).

Message passing on 2-sets follows the neighbourhood definition from Equation (3). For a 2-set $s = \{u, v\}$, the local neighbourhood decomposes into two cases:

$$\mathcal{N}_L(\{u, v\}) = \underbrace{\{\{v, w\} : w \neq u,\ (u, w) \in E\}}_{\text{Case 1: replace } u \text{ with } w} \cup \underbrace{\{\{u, w\} : w \neq v,\ (v, w) \in E\}}_{\text{Case 2: replace } v \text{ with } w}. \tag{9}$$

In Case 1, we remove u from the pair and add a new node w that must be adjacent to u in the original graph. In Case 2, we remove v and add w adjacent to v. This ensures that neighbourhood relationships in the 2-GNN reflect the edge structure of the underlying graph.

The 2-GNN layer update rule mirrors the 1-GNN structure:

$$\mathbf{f}_s^{(\ell+1)} = \sigma\left(\mathbf{f}_s^{(\ell)}\mathbf{W}_3^{(\ell)} + \sum_{s' \in \mathcal{N}_L(s)} \mathbf{f}_{s'}^{(\ell)}\mathbf{W}_4^{(\ell)}\right), \tag{10}$$

where $\mathbf{W}_3^{(\ell)}, \mathbf{W}_4^{(\ell)}$ are the 2-GNN weight matrices at layer ℓ. We apply $T_2 = 2$ layers.

The **hierarchical 1-2-GNN** combines both levels. It first runs the 1-GNN (Equation 4) for $T_1 = 3$ layers to obtain node embeddings, then uses these embeddings to initialise 2-GNN features (Equation 8), and runs $T_2 = 2$ layers of 2-GNN message passing. The final graph representation concatenates both readouts:

$$\mathbf{h}_G = \Big[\mathbf{h}_G^{(1)} \parallel \mathbf{h}_G^{(2)}\Big] \in \mathbb{R}^{2d_T}, \tag{11}$$

where the 1-GNN readout is $\mathbf{h}_G^{(1)} = \sum_{v \in V} \mathbf{h}_v^{(T_1)}$ and the 2-GNN readout pools over all pairs:

$$\mathbf{h}_G^{(2)} = \sum_{\{u,v\} \in \binom{V}{2}} \mathbf{f}_{\{u,v\}}^{(T_2)} \in \mathbb{R}^{d_T}. \tag{12}$$

The concatenated embedding $\mathbf{h}_G$ is then fed through the same classifier structure as Equation (7), but with input dimension $2d_T$ instead of d_T.

Scalability. The number of 2-sets grows as $\binom{n}{2} = \frac{n(n-1)}{2}$, which becomes prohibitive for large graphs. For PROTEINS (up to 620 nodes, yielding up to 191,890 pairs), we process all pairs using a numba-accelerated sparse kernel with CSR adjacency representation, achieving efficient $O(\text{degree})$ neighbour lookup instead of $O(N)$ dense masks. This enables exact computation over the full pairwise structure without sampling.

3.3 GIN-Graph Generator

The generator $\mathcal{G}$ maps a latent noise vector $\mathbf{z} \in \mathbb{R}^{d_z}$ to a graph $(\tilde{A}, \tilde{X})$ through a backbone MLP followed by separate output heads:

$$\mathbf{h} = \text{LeakyReLU}(\mathbf{W}_2 \, \text{LeakyReLU}(\mathbf{W}_1 \mathbf{z})) \in \mathbb{R}^{2d_h}, \tag{13}$$

$$\tilde{A}_{\text{raw}} = \text{sym}(\text{reshape}(\mathbf{h}\,\mathbf{W}_A,\ N \times N)), \tag{14}$$

$$\tilde{X}_{\text{raw}} = \text{reshape}(\mathbf{h}\,\mathbf{W}_X,\ N \times D), \tag{15}$$

where $d_z = 32$ is the latent dimension, $d_h = 128$ is the hidden dimension, N is the maximum number of nodes, D is the number of node feature types, and $\text{sym}(\mathbf{M}) = \frac{1}{2}(\mathbf{M} + \mathbf{M}^\top)$ ensures the adjacency logits are symmetric (undirected graphs).

Gumbel-Softmax. Graphs are inherently discrete objects: edges are either present or absent, and node types are categorical. Standard gradient-based optimisation cannot backpropagate through discrete sampling. The Gumbel-Softmax trick [Jang et al.(2017)] provides a differentiable relaxation.

For a categorical distribution with unnormalised log-probabilities (logits) $\pi_1, \ldots, \pi_K$, the Gumbel-Softmax sample is:

$$y_i = \frac{\exp((\pi_i + g_i)/\tau)}{\sum_{j=1}^{K} \exp((\pi_j + g_j)/\tau)}, \quad g_i = -\log(-\log(u_i)), \quad u_i \sim \text{Uniform}(0,1), \tag{16}$$

where $\tau > 0$ is a temperature parameter. As $\tau \to 0$, the output approaches a one-hot vector (hard categorical sample); as $\tau \to \infty$, it approaches a uniform distribution. During training, we use the *straight-through estimator*: the forward pass produces hard (discrete) samples via $\arg\max$, but the backward pass uses the continuous Gumbel-Softmax gradients. This gives the generator discrete outputs while maintaining gradient flow.

For the **adjacency matrix**, each potential edge (i, j) is a binary choice (edge/no-edge). We construct 2-class logits $[\tilde{A}_{\text{raw}}[i,j],\ -\tilde{A}_{\text{raw}}[i,j]]$ and apply Gumbel-Softmax with $K = 2$. To guarantee symmetry ($\tilde{A}_{ij} = \tilde{A}_{ji}$), we sample only the upper triangle and mirror it:

$$\tilde{A}_{\text{upper}} = \text{triu}(\text{GumbelSoftmax}(\tilde{A}_{\text{raw}}, \tau)), \quad \tilde{A} = \tilde{A}_{\text{upper}} + \tilde{A}_{\text{upper}}^\top. \tag{17}$$

For **node features**, each node has D possible types (e.g., 7 atom types for MUTAG). We apply Gumbel-Softmax with $K = D$ to produce one-hot feature vectors:

$$\tilde{X}[v,:] = \text{GumbelSoftmax}(\tilde{X}_{\text{raw}}[v,:],\ \tau) \in \{0,1\}^D. \tag{18}$$

The temperature $\tau = 1.0$ during training (allowing exploration) and $\tau = 0.1$ during evaluation (producing sharper, more discrete outputs).

3.4 Training Objective

The total generator loss combines adversarial, GNN-guidance, and structural regularisation terms:

$$\mathcal{L}_G = (1 - \lambda(t))\,\mathcal{L}_{\text{GAN}} + \lambda(t)\,\mathcal{L}_{\text{GNN}} + \mathcal{L}_{\text{reg}}, \tag{19}$$

where $\lambda(t) \in [0, 1]$ is a dynamic weight that increases over training, and $\mathcal{L}_{\text{reg}}$ comprises structural regularisation losses described below.

WGAN-GP loss. We use a Wasserstein GAN with gradient penalty [Gulrajani et al.(2017)]. The discriminator $\mathcal{D}$ (a GNN-based network) estimates the Wasserstein distance between real and generated graph distributions. The discriminator loss is:

$$\mathcal{L}_D = \underbrace{\mathbb{E}_{\tilde{G}\sim\mathcal{G}}[\mathcal{D}(\tilde{G})]}_{\text{score on fake}} - \underbrace{\mathbb{E}_{G\sim p_{\text{data}}}[\mathcal{D}(G)]}_{\text{score on real}} + \underbrace{\lambda_{\text{GP}}\,\mathbb{E}_{\hat{G}}\Big[(\|\nabla_{\hat{G}}\mathcal{D}(\hat{G})\|_2 - 1)^2\Big]}_{\text{gradient penalty}}, \tag{20}$$

where $\hat{G} = \epsilon G + (1-\epsilon)\tilde{G}$ with $\epsilon \sim \text{Uniform}(0, 1)$ is a random interpolation, and $\lambda_{\text{GP}} = 10$. The gradient penalty enforces the Lipschitz constraint on $\mathcal{D}$, stabilising training. The generator's adversarial loss is simply:

$$\mathcal{L}_{\text{GAN}} = -\mathbb{E}_{\tilde{G}\sim\mathcal{G}}[\mathcal{D}(\tilde{G})]. \tag{21}$$

GNN guidance loss. We maximise the pretrained classifier's confidence on the target class c:

$$\mathcal{L}_{\text{GNN}} = -\log p_\Phi(y = c \mid \tilde{G}), \tag{22}$$

where $p_\Phi(y = c \mid \tilde{G})$ is the softmax probability assigned to class c by the pretrained k-GNN Φ. Minimising this loss pushes the generator to produce graphs that the classifier recognises as belonging to class c.

Dynamic weighting. The balance parameter $\lambda(t)$ follows a sigmoid schedule [Sun et al.(2025)]:

$$\lambda(t) = \lambda_{\min} + (\lambda_{\max} - \lambda_{\min}) \cdot \sigma\left(k \cdot \left(\frac{2(t/T - p)}{1 - p} - 1\right)\right), \tag{23}$$

where T is the total number of training iterations, $p = 0.4$ is the fraction of training before λ begins increasing, $k = 10$ controls the steepness of the transition, and $\sigma(x) = 1/(1+e^{-x})$ is the logistic sigmoid.

The intuition is as follows. When $t \ll pT$, the sigmoid argument is strongly negative, giving $\lambda(t) \approx 0$: the generator focuses on learning realistic graph structure (GAN loss dominates). As t passes pT, $\lambda(t)$ increases rapidly toward 1: the generator shifts to producing class-specific patterns (GNN loss dominates). Without this schedule, the generator would collapse to structurally degenerate graphs that fool the classifier but look nothing like real molecules or proteins.

Structural regularisation. The generator outputs a fixed-size $N \times N$ adjacency matrix, but real graphs vary in size. Without explicit constraints, the Gumbel-Softmax sampling

tends to activate most node slots, producing graphs much larger than the real class mean. We add two regularisation terms:

$$\mathcal{L}_{\text{reg}} = \lambda_{\text{size}} \cdot \left(\frac{\hat{n} - \bar{n}_c}{\bar{n}_c} \right)^2 + \lambda_{\text{type}} \cdot \sum_{i=1}^{D} (\hat{p}_i - p_i^c)^2 \,, \tag{24}$$

where $\hat{n} = \sum_v \sigma(10 \cdot (d_v - 0.5))$ is a differentiable soft count of active nodes (using the sigmoid of the degree d_v minus a threshold), $\bar{n}_c$ is the mean node count for class c, $\hat{p}_i$ is the generated frequency of node type i, and p_i^c is the real frequency in class c. The size loss ($\lambda_{\text{size}} = 10$) penalises deviation from the class mean graph size, and the node type loss ($\lambda_{\text{type}} = 1$) encourages the generated node type distribution to match the real class distribution.

3.5 Dense Wrapper for Differentiable k-GNN Inference

A key technical challenge is that the pretrained k-GNN uses sparse PyTorch Geometric message-passing operations (scatter-add on edge lists), while the generator outputs dense adjacency matrices $\tilde{A} \in [0,1]^{N \times N}$ that require gradient flow for backpropagation. We implement a fully differentiable *dense wrapper* that re-implements the k-GNN forward pass using dense matrix operations while reusing the pretrained weights (which are frozen).

Dense 1-GNN. The node-level update (Equation 5) translates directly to dense batched computation. For a batch of B graphs with node features $\mathbf{X} \in \mathbb{R}^{B \times N \times d}$ and adjacency $\mathbf{A} \in [0,1]^{B \times N \times N}$:

$$\mathbf{H}^{(t)} = \sigma\Big(\mathbf{H}^{(t-1)} \mathbf{W}_1^{(t)} + \mathbf{A}\, \mathbf{H}^{(t-1)} \mathbf{W}_2^{(t)} \Big) \,, \tag{25}$$

where the batch dimension is broadcast. The matrix product $\mathbf{A}\,\mathbf{H}^{(t-1)}$ replaces the sparse scatter-add operation and is fully differentiable with respect to the continuous $\mathbf{A}$. Crucially, the weights $\mathbf{W}_1^{(t)}, \mathbf{W}_2^{(t)}$ are copied from the pretrained sparse model and kept frozen.

Dense 2-GNN. The 2-GNN component requires constructing pair features and aggregating over 2-set neighbourhoods, both as dense operations. We represent all $\binom{N}{2}$ pair features as an $N \times N$ tensor (using only the upper triangle for the canonical pairs).

First, we construct the pair features from the 1-GNN node embeddings $\mathbf{H}^{(T_1)} \in \mathbb{R}^{B \times N \times d_T}$:

$$\mathbf{F}[i,j] = \big[\mathbf{h}_{\min(i,j)} \parallel \mathbf{h}_{\max(i,j)} \parallel A_{ij} \big] \in \mathbb{R}^{2d_T+1}, \tag{26}$$

where the canonical ordering $(\min, \max)$ ensures that $\mathbf{F}[i,j] = \mathbf{F}[j,i]$, matching the unordered pair semantics. The isomorphism type A_{ij} is the continuous adjacency value from the generator, maintaining differentiability.

For the neighbourhood aggregation, recall from Equation (9) that there are two cases. Using the transformed features $\mathbf{g} = \mathbf{F}\,\mathbf{W}_4^{(\ell)} \in \mathbb{R}^{B \times N \times N \times d_T}$, the aggregation for pair (i,j) is:

$$\text{agg}_1[i,j] = \sum_{\substack{w=1 \\ w \neq j}}^{N} A_{iw} \cdot \mathbf{g}[j,w] \qquad \text{(replace } i \text{ with } w \text{ adjacent to } i\text{)}, \tag{27}$$

$$\text{agg}_2[i,j] = \sum_{\substack{w=1 \\ w \neq i}}^{N} A_{jw} \cdot \mathbf{g}[i,w] \qquad \text{(replace } j \text{ with } w \text{ adjacent to } j\text{)}. \tag{28}$$

The constraints $w \neq j$ in agg_1 and $w \neq i$ in agg_2 are essential: without them, the summation includes the self-pair $\{j, j\}$ or $\{i, i\}$, which has cardinality 1 and is not a valid 2-set. The self-loop exclusion ($A_{ii} = 0$) already removes $w = i$ from agg_1 and $w = j$ from agg_2, but the opposite boundary terms must be explicitly masked. In practice, we zero out the diagonal of $\mathbf{g}$ (setting $\mathbf{g}[k, k] = \mathbf{0}$ for all k) before the summation, which eliminates both boundary terms simultaneously.

These sums are computed efficiently via Einstein summation (`einsum`) over the masked $\mathbf{g}$. The key insight is that multiplying by A_{iw} (a continuous value from the generator) acts as a soft attention over potential neighbours, and gradients flow through $\mathbf{A}$ into the generator. The full 2-GNN layer update is then:

$$\mathbf{F}^{(\ell+1)} = \sigma\Big(\mathbf{F}^{(\ell)}\,\mathbf{W}_3^{(\ell)} + \text{agg}_1^{(\ell)} + \text{agg}_2^{(\ell)}\Big). \tag{29}$$

After T_2 layers, the 2-GNN readout sums over the upper-triangular entries:

$$\mathbf{h}_G^{(2)} = \sum_{i<j} \mathbf{F}^{(T_2)}[i, j]. \tag{30}$$

Both the 1-GNN and 2-GNN dense paths are fully differentiable through $\mathbf{A}$, enabling the generator to learn how edge structure affects the classifier's output at both the node and pair level.

3.6 Validation Score

We evaluate generated explanations using a composite validation score [Sun et al.(2025)]:

$$v = (s \cdot p \cdot d)^{1/3}, \tag{31}$$

comprising three components. The geometric mean ensures that all three aspects must be satisfied: a graph cannot achieve a high score if it fails on any single component.

Embedding similarity (s). The cosine similarity between the generated graph's embedding (computed via the dense wrapper) and a *class centroid*—the mean embedding of all real graphs in the target class, computed via the sparse k-GNN on the training set:

$$s = \cos(\mathbf{h}_{\tilde{G}},\, \boldsymbol{\mu}_c) = \frac{\mathbf{h}_{\tilde{G}} \cdot \boldsymbol{\mu}_c}{\|\mathbf{h}_{\tilde{G}}\|\,\|\boldsymbol{\mu}_c\|}, \quad \text{where } \boldsymbol{\mu}_c = \frac{1}{|\mathcal{G}_c|} \sum_{G \in \mathcal{G}_c} \mathbf{h}_G. \tag{32}$$

This measures whether the generated graph "looks like" a real class member in the model's learned representation space. The centroid $\boldsymbol{\mu}_c$ is computed once and stored in the GIN-Graph checkpoint.

Note on the corrected metric. The original GIN-Graph implementation used the prediction probability p as a proxy for embedding similarity (effectively setting $s = p$), which collapsed the validation score to $v = (p^2 \cdot d)^{1/3}$. We corrected this by computing actual cosine similarity via the dense wrapper's `get_embedding()` method.

Prediction probability (p). The softmax probability assigned to the target class by the pretrained classifier: $p = p_\Phi(y = c \mid \tilde{G})$.

Degree score (d). A Gaussian kernel measuring how realistic the graph's average degree is relative to the target class:

$$d = \exp\left(-\frac{(\bar{d}_{\tilde{G}} - \mu_d)^2}{2\sigma_d^2}\right), \tag{33}$$

where $\bar{d}_{\tilde{G}} = 2|\tilde{E}|/|\tilde{V}|$ is the average degree of the generated graph, and μ_d, σ_d are the mean and standard deviation of average degrees across real graphs in the target class. A graph with average degree close to the class mean receives $d \approx 1$; one with atypical degree is penalised exponentially.

A generated graph is considered **valid** if its average degree falls within 3 standard deviations of the class mean and its validation score exceeds 0.5.

Granularity (κ). We additionally report:

$$\kappa = 1 - \min\left(1,\ \frac{n_{\tilde{G}}}{\bar{n}_c}\right), \tag{34}$$

where $n_{\tilde{G}}$ is the number of active (non-isolated) nodes in the explanation and $\bar{n}_c$ is the average number of nodes in class c. Values near 0 indicate coarse-grained explanations (graph-sized); values near 1 would indicate fine-grained substructure highlights.

4 Experimental Setup

4.1 Datasets

We evaluate on two standard graph classification benchmarks (Table 1).

Table 1: Dataset statistics. GIN Gen is the maximum number of nodes used for explanation generation.

Dataset	Graphs	Node Feats	Max Nodes	GIN Gen	Task
MUTAG	188	7 (atom types)	28	28	Mutagenicity
PROTEINS	1113	3 (sec. struct.)	620	50	Enzyme vs. non-enzyme

MUTAG [Debnath et al.(1991)] contains 188 molecular graphs labelled by mutagenic effect on *Salmonella typhimurium*. Nodes represent atoms (C, N, O, F, I, Cl, Br) and edges represent chemical bonds. The two classes are *Mutagen* (class 0, 125 graphs) and *Non-Mutagen* (class 1, 63 graphs).

PROTEINS [Borgwardt et al.(2005)] contains 1113 protein graphs where nodes are secondary structure elements (helix, sheet, coil/turn) connected if they are neighbours in the amino acid sequence or within a distance threshold in 3D space. The classes are *Non-Enzyme* (class 0) and *Enzyme* (class 1). For GIN-Graph generation, we cap the graph size at 50 nodes (the median protein graph has $\sim$26 nodes), as the explanations highlight key substructures rather than reproducing entire large graphs.

4.2 Model Configuration

All k-GNN models use a hidden dimension of $d_T = 64$. The 1-GNN uses $T_1 = 3$ message-passing layers; the 1-2-GNN uses 3 layers for the 1-GNN component and $T_2 = 2$ layers for the 2-GNN component. Both use dropout of 0.5 and are trained with Adam (lr = 0.01) for 100 epochs with cross-entropy loss. Data is split 80/20 into train/test with a fixed seed of 42.

The GIN-Graph generator uses a latent dimension $d_z = 32$, hidden dimension $d_h = 128$, and is trained for 300 epochs with Adam (lr = 0.001) and batch size 64. WGAN-GP uses $\lambda_{\text{GP}} = 10$ and trains the discriminator once per generator update ($n_{\text{critic}} = 1$). The dynamic weighting uses $p = 0.4$, $k = 10$, $\lambda_{\min} = 0$, $\lambda_{\max} = 1$.

Computational cost. Table 2 reports wall-clock times on CPU. The 2-GNN component incurs substantial overhead due to the $O(n^2)$ pair construction and message passing. On PROTEINS, the 1-2-GNN is $\sim$326$\times$ slower per forward pass than the 1-GNN, reflecting the cost of processing all $\binom{n}{2}$ node pairs for graphs with up to 620 nodes. GIN-Graph generation uses the dense wrapper, where the 1-2-GNN overhead is $\sim$36–74$\times$ at the fixed generation size ($N = 28$ for MUTAG, $N = 50$ for PROTEINS).

Table 2: Wall-clock inference times on CPU. k-GNN: sparse forward pass (batch of 64 graphs). GIN-Graph: dense wrapper generation (100 graphs).

Dataset	Model	k-GNN (ms/batch)	GIN-Graph gen (s/100)
MUTAG	1-GNN	1.9	0.02
MUTAG	1-2-GNN	99.5 (52×)	1.48 (74×)
PROTEINS	1-GNN	6.8	0.04
PROTEINS	1-2-GNN	2214 (326×)	1.43 (36×)

4.3 Evaluation Protocol

For each model-dataset-class combination, we generate 100 explanation graphs at evaluation temperature $\tau = 0.1$ and evaluate them using the validation score (Equation 31). We report:

- **Validity rate**: fraction of explanations passing degree and score thresholds;
- **Mean validation score**: averaged over all 100 generated graphs;
- **Component scores**: embedding similarity (s), prediction probability (p), degree score (d);
- **Granularity**: how fine-grained the explanations are.

5 Results

5.1 k-GNN Classification Performance

Table 3 summarises the classification results. On MUTAG, both models achieve identical best test accuracy of 89.5%, indicating that the additional pairwise message passing does

not provide a classification advantage on this small molecular dataset. On PROTEINS, both models achieve comparable classification accuracy (79.8% vs. 78.9%) after 100 epochs of training, with the 1-2-GNN now processing all $\binom{n}{2}$ node pairs via a numba-accelerated kernel instead of sampling a subset. The different best epochs on PROTEINS (120 for 1-GNN, 75 for 1-2-GNN) suggest different training dynamics once full pairwise structure is included, but under a single split this should be read as descriptive rather than conclusive.

Table 3: k-GNN classification results. Best test accuracy reported over all epochs.

Dataset	**Model**	**Params**	**Final Test Acc**	**Best Acc**	**Best Epoch**
MUTAG	1-GNN	21,570	81.6%	89.5%	84
MUTAG	1-2-GNN	50,370	76.3%	89.5%	18
PROTEINS	1-GNN	21,058	75.8%	79.8%	120
PROTEINS	1-2-GNN	49,858	73.1%	78.9%	75

5.2 GIN-Graph Explanation Quality: MUTAG

Table 4 presents the GIN-Graph results on MUTAG. All configurations were fully trained for 300 epochs, enabling fair cross-model comparison on both classes.

Table 4: GIN-Graph explanation quality on MUTAG. All models trained for 300 epochs.

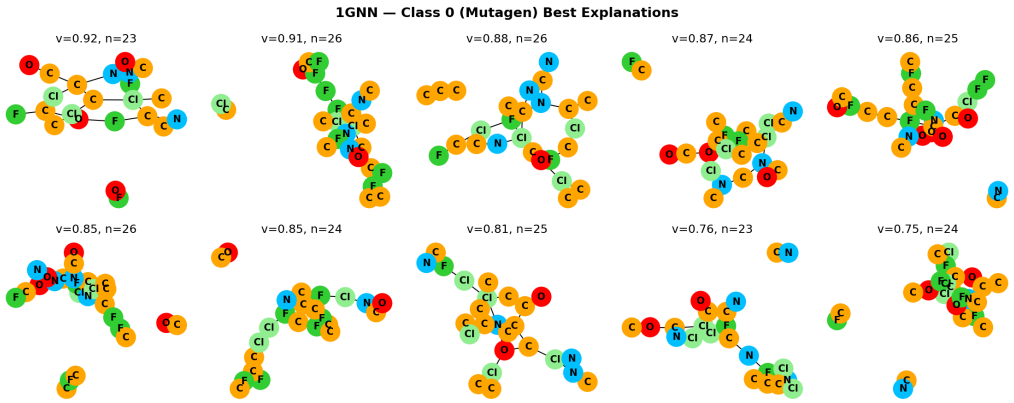
Class	**Model**	**Valid%**	**Val Score**	s	p	d
0 (Mutagen)	1-GNN	36%	0.373	0.701	1.000	0.260
0 (Mutagen)	1-2-GNN	**41%**	**0.434**	**0.771**	1.000	**0.314**
1 (Non-Mutagen)	1-GNN	78%	0.681	**0.935**	1.000	0.490
1 (Non-Mutagen)	1-2-GNN	**82%**	**0.735**	0.933	1.000	**0.565**

Both models achieve successful generation on both classes, with the 1-2-GNN showing somewhat higher validity rates (82% vs. 78% on class 1, 41% vs. 36% on class 0) and validation scores (0.735 vs. 0.681 on class 1). However, under the single-seed setup these quantitative gaps should be treated cautiously (Section 7). The more important result is the *qualitative divergence* in what the two models generate.

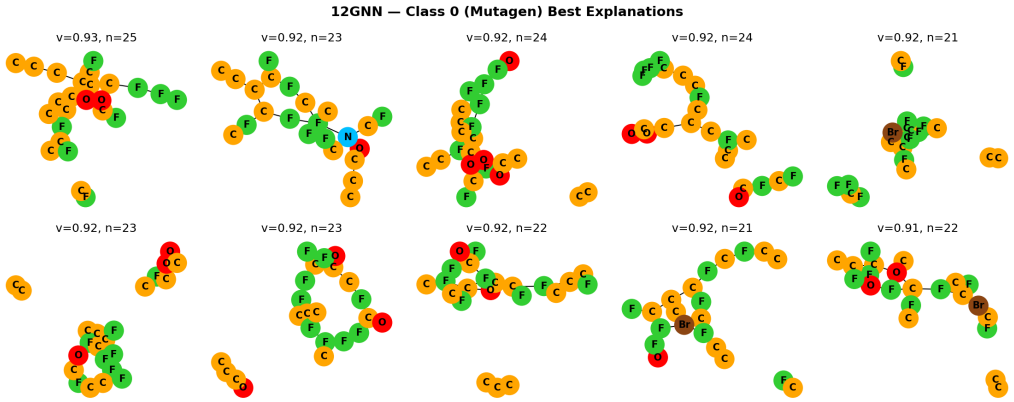
Despite identical classification accuracy (89.5%), the two models produce strikingly different class-0 explanation families (Figure 1). Across the validation-ranked top explanation set, the 1-GNN remains C/N/O-dominated (42.9% C, 13.9% N, 13.2% O), while the 1-2-GNN shifts toward a much more halogen-heavy pattern (31.1% F, 1.1% Br) with almost no nitrogen (0.4% N). Because these percentages come from the top-ranked set rather than the full generated population, they should be read as representative high-scoring prototypes. The population-level comparison in Section 5.6 shows the same directional pattern: on MUTAG class 0, the 1-2-GNN generator overproduces halogens much more strongly than the 1-GNN.

Class 0 remains the harder class for both models, but this should not be read only as a failure case. MUTAG mutagens are structurally diverse, with nitro/amino-rich and halogen-rich signatures both appearing in the data. The contrast between the 1-GNN's

N/O-rich prototypes and the 1-2-GNN's halogen-rich prototypes therefore looks less like contradictory noise and more like evidence that the two architectures settle on different mutagenic signature families within a heterogeneous class. Both models still achieve perfect prediction probability ($p = 1.0$).



(a) 1-GNN explanations for Mutagen



(b) 1-2-GNN explanations for Mutagen

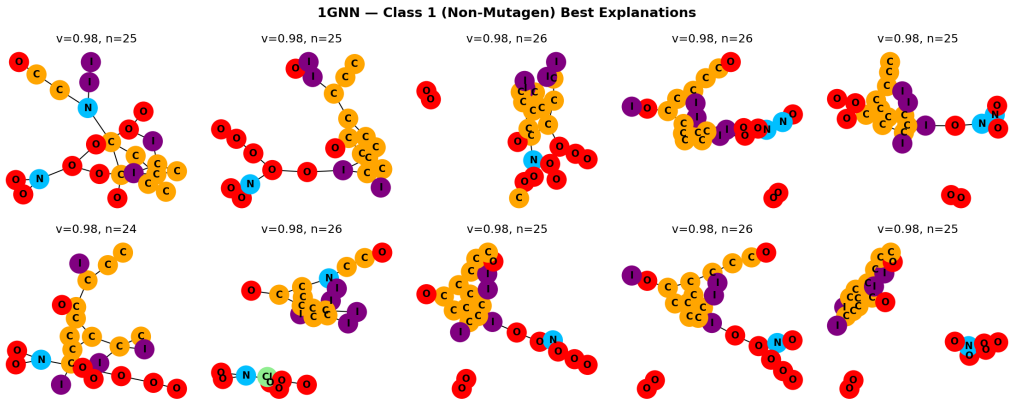
Figure 1: Top-ranked GIN-Graph explanations on MUTAG class 0 (Mutagen). The 1-GNN repeatedly produces mixed C/N/O scaffolds with scattered halogens, whereas the 1-2-GNN collapses onto narrower fluorine- and bromine-heavy prototype families. Node colours indicate atom types (see Figure 3).

Figure 2 shows the top-ranked explanation graphs for both models on MUTAG class 1.

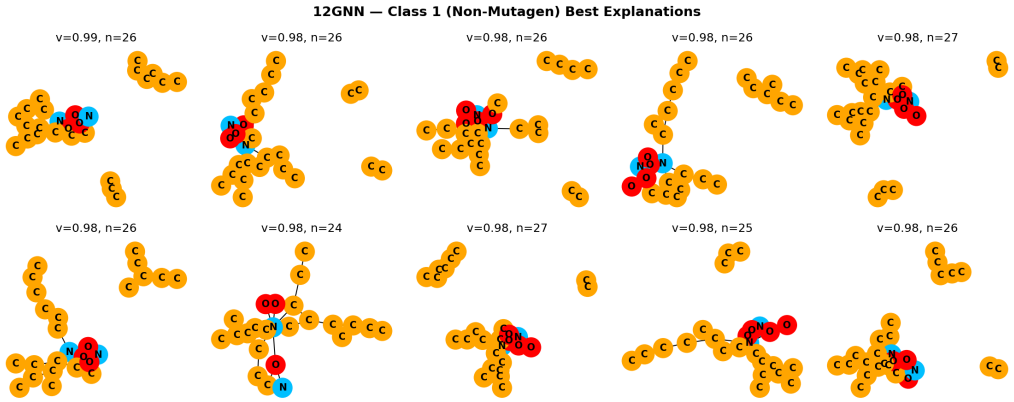
Both model families achieve near-perfect top validation scores (0.984–0.986), but the representative class-1 structures differ noticeably. The 1-GNN frequently produces longer oxygen- and iodine-bearing branches attached to a carbon scaffold, whereas the 1-2-GNN concentrates more strongly on compact carbon-dominated cores with small N/O motifs. As in class 0, these top-set percentages are conditioned on validation ranking; Table 8 in Section 5.6 provides the broader population-level composition.

5.3 GIN-Graph Explanation Quality: PROTEINS

Table 5 presents results on PROTEINS. All configurations were fully trained for 300 epochs with graph size and node type regularisation (see Section 3.4). Unlike MUTAG,



(a) 1-GNN explanations for Non-Mutagen



(b) 1-2-GNN explanations for Non-Mutagen

Figure 2: Top-ranked GIN-Graph explanations on MUTAG class 1 (Non-Mutagen). The 1-GNN explanations often attach oxygen- and iodine-bearing branches to a carbon backbone, while the 1-2-GNN explanations are more compact and carbon-dominated with smaller N/O motifs. Node colours indicate atom types (see Figure 3).

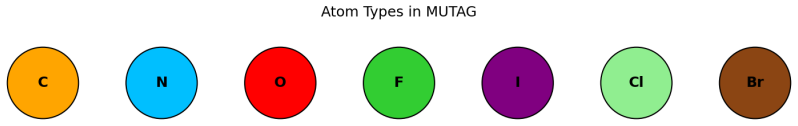


Figure 3: MUTAG atom type colour legend. Atoms: C (orange), N (blue), O (red), F (green), I (purple), Cl (light green), Br (brown).

the two models show complementary strengths, with each achieving higher scores on a different class.

Table 5: GIN-Graph explanation quality on PROTEINS. All models trained for 300 epochs with structural regularisation.

Class	**Model**	**Valid%**	**Val Score**	s	p	d
0 (Non-Enzyme)	1-GNN	100%	0.986	0.987	0.986	0.984
0 (Non-Enzyme)	1-2-GNN	100%	0.760	0.782	0.564	0.996
1 (Enzyme)	1-GNN	37%	0.330	0.356	0.919	0.837
1 (Enzyme)	1-2-GNN	94%	0.524	0.151	0.998	0.961

The 1-GNN achieves near-perfect scores on class 0 (Non-Enzyme) across all components ($v = 0.986$, $s = 0.987$, $p = 0.986$), while the 1-2-GNN achieves lower but still functional prediction probability ($p = 0.564$) and overall validation score ($v = 0.760$). On class 1 (Enzyme), both models achieve high prediction probability ($p = 0.919$ and 0.998), but the 1-GNN struggles with validity (37%) while the 1-2-GNN reaches 94% validity. Notably, both models show low embedding similarity on Enzyme ($s = 0.356$ and 0.151), indicating that correct classification alone does not imply close alignment with the real Enzyme centroid.

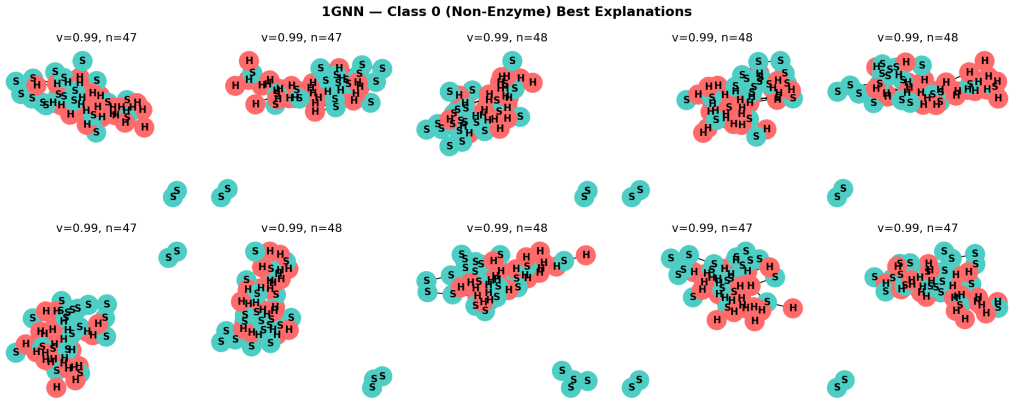
Both models achieve comparable classification accuracy (79.8% vs. 78.9%), so this complementary pattern reflects genuine differences in what the two architectures learn about protein structure rather than a simple classifier quality gap. The result is mixed rather than one-sided: the 1-GNN is strongest on Non-Enzyme by the original validation score, while the 1-2-GNN is strongest on Enzyme validity and later proves structurally closer on the population-level Non-Enzyme node-type distribution.

The representative examples make the class split concrete. The 1-GNN Non-Enzyme explanations are consistently large and compositionally balanced, matching the near-perfect class-0 metrics, whereas its Enzyme outputs are much less stable and include both compact plausible motifs and looser elongated structures, consistent with the 37% validity rate. The 1-2-GNN shows the opposite profile: its Non-Enzyme explanations are extremely regular, while its Enzyme explanations repeatedly collapse onto sheet-heavy cores. Taken together, the figures support a model-specific prototype story rather than a universal winner.

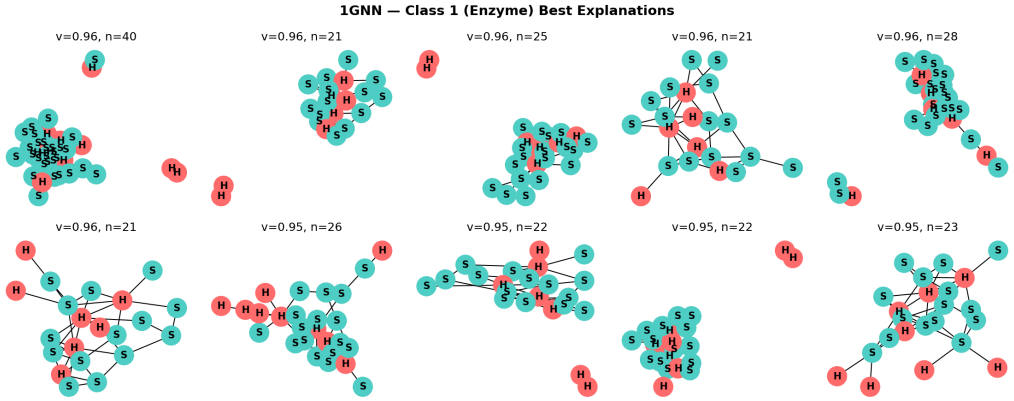
5.4 Metric Decomposition

Table 6 provides a detailed comparison of the three validation score components across all configurations, now with all models trained for 300 epochs.

On MUTAG, both models achieve perfect prediction probability ($p = 1.0$) on both classes, indicating that both classifiers provide strong gradient signal to the generators. The degree score shows the largest variation: the 1-2-GNN produces graphs with more realistic average degree on both classes (0.565 vs. 0.490 on class 1, 0.314 vs. 0.260 on class 0). Embedding similarity is comparable on class 1 ($s \approx 0.93$) and somewhat higher for the 1-2-GNN on class 0 (0.771 vs. 0.701).



(a) 1-GNN explanations for Non-Enzyme

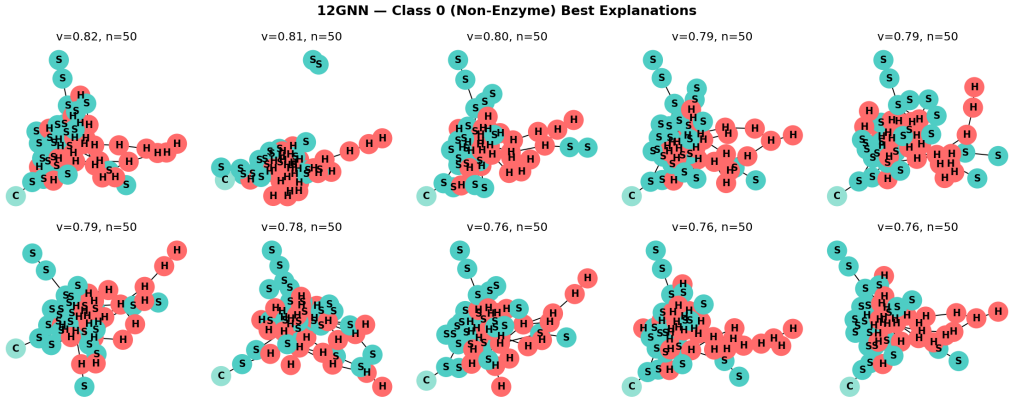


(b) 1-GNN explanations for Enzyme

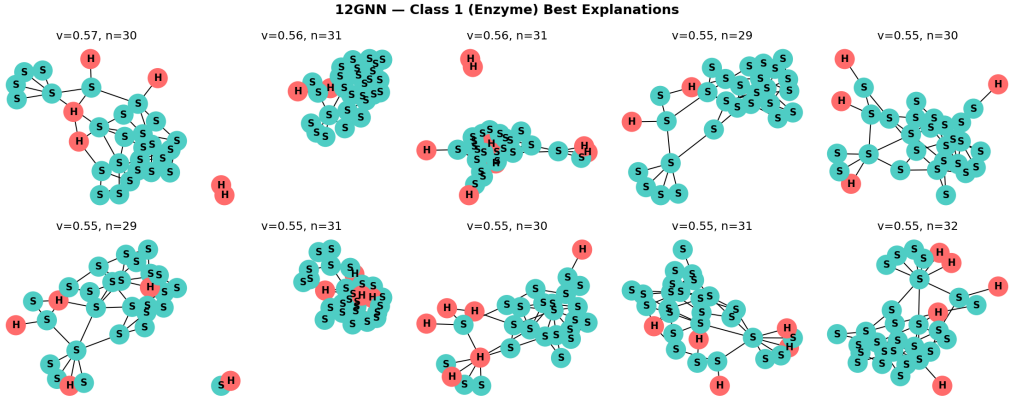
Figure 4: Top-ranked GIN-Graph explanations on PROTEINS (1-GNN). The Non-Enzyme explanations are large balanced H/S structures near the 50-node cap, while the Enzyme explanations are more variable in size and shape. Node colours indicate secondary structure types: helix (H, pink), sheet (S, teal), coil/turn (C, green).

Table 6: Validation score decomposition for all GIN-Graph models (300 epochs).

Dataset	**Class**	**Model**	s (emb. sim)	p (pred. prob)	d (degree)
MUTAG	0 (Mutagen)	1-GNN	0.701	1.000	0.260
MUTAG	0 (Mutagen)	1-2-GNN	0.771	1.000	0.314
MUTAG	1 (Non-Mut.)	1-GNN	0.935	1.000	0.490
MUTAG	1 (Non-Mut.)	1-2-GNN	0.933	1.000	0.565
PROTEINS	0 (Non-Enz.)	1-GNN	0.987	0.986	0.984
PROTEINS	0 (Non-Enz.)	1-2-GNN	0.782	0.564	0.996
PROTEINS	1 (Enzyme)	1-GNN	0.356	0.919	0.837
PROTEINS	1 (Enzyme)	1-2-GNN	0.151	0.998	0.961



(a) 1-2-GNN explanations for Non-Enzyme



(b) 1-2-GNN explanations for Enzyme

Figure 5: Top-ranked GIN-Graph explanations on PROTEINS (1-2-GNN). The Non-Enzyme explanations are highly regular 50-node H/S structures, whereas the Enzyme explanations repeatedly collapse onto sheet-dominated cores with smaller helix appendages. Class 0 explanations achieve validation scores $v = 0.76$–0.82 with 50 nodes, while class 1 explanations achieve $v = 0.52$–0.58 with 29–32 nodes.

On PROTEINS, the metric profiles reveal how each architecture interacts differently with the generator. The 1-GNN achieves near-perfect scores on class 0 ($s = 0.987$, $p = 0.986$, $d = 0.984$) but lower embedding similarity on class 1 ($s = 0.356$), indicating generated Enzyme graphs diverge from the real embedding centroid despite being correctly classified ($p = 0.919$). The 1-2-GNN achieves moderate prediction probability on class 0 ($p = 0.564$) with high embedding similarity ($s = 0.782$) and near-perfect degree score ($d = 0.996$), while on class 1 it achieves near-perfect prediction probability ($p = 0.998$) but very low embedding similarity ($s = 0.151$). The degree scores are generally high across PROTEINS ($d > 0.83$), confirming that structural realism is maintained by the WGAN-GP component.

5.5 Training Dynamics

Figure 6 shows a representative training curve from the MUTAG 1-2-GNN class 1 experiment. The three loss curves illustrate the interplay between the WGAN-GP adversarial training and the GNN guidance objective over 300 epochs.

During the first $\sim$120 epochs ($\lambda \approx 0$), the generator loss and discriminator loss dominate: the generator learns to produce structurally realistic graphs that fool the discriminator, without pressure to target a specific class. As the dynamic weighting $\lambda(t)$ increases (Equation 23), the GNN guidance loss takes over, steering the generator toward class-specific patterns. This transition is visible as the GNN loss drops sharply while the adversarial losses plateau.

The training behaviour varies across configurations. On MUTAG, all four generators converge smoothly, consistent with the small graph sizes ($N = 28$) and low feature dimensionality (7 atom types). On PROTEINS, the training dynamics are more complex: the 1-2-GNN class 0 generator achieves moderate prediction probability ($p = 0.564$ at evaluation), lower than the 1-GNN's near-perfect alignment ($p = 0.986$) on the same class. This gap reflects the difficulty of optimising through the 1-2-GNN's higher-dimensional embedding space ($\mathbb{R}^{128}$ vs. $\mathbb{R}^{64}$), where the generator must satisfy both node-level and pair-level classifier components simultaneously.

5.6 Generated Dataset Comparison

To evaluate the fidelity and cross-model transferability of generated explanations at scale, we generate 500 graphs per class from each GIN-Graph generator and compare the resulting synthetic datasets against the real data. This population-level analysis complements the per-graph metrics reported in Sections 5.2–5.4 by revealing systematic differences between what each model has learned. No retraining is performed—all graphs are generated from the existing checkpoints.

5.6.1 Structural Fidelity

Table 7 summarises the structural comparison. Across both datasets, degree distributions are matched more reliably than graph-size distributions. Kolmogorov–Smirnov statistics on degree range from 0.09 to 0.21, with generated mean degrees remaining close to the real values in every configuration. By contrast, the size KS statistics are substantially

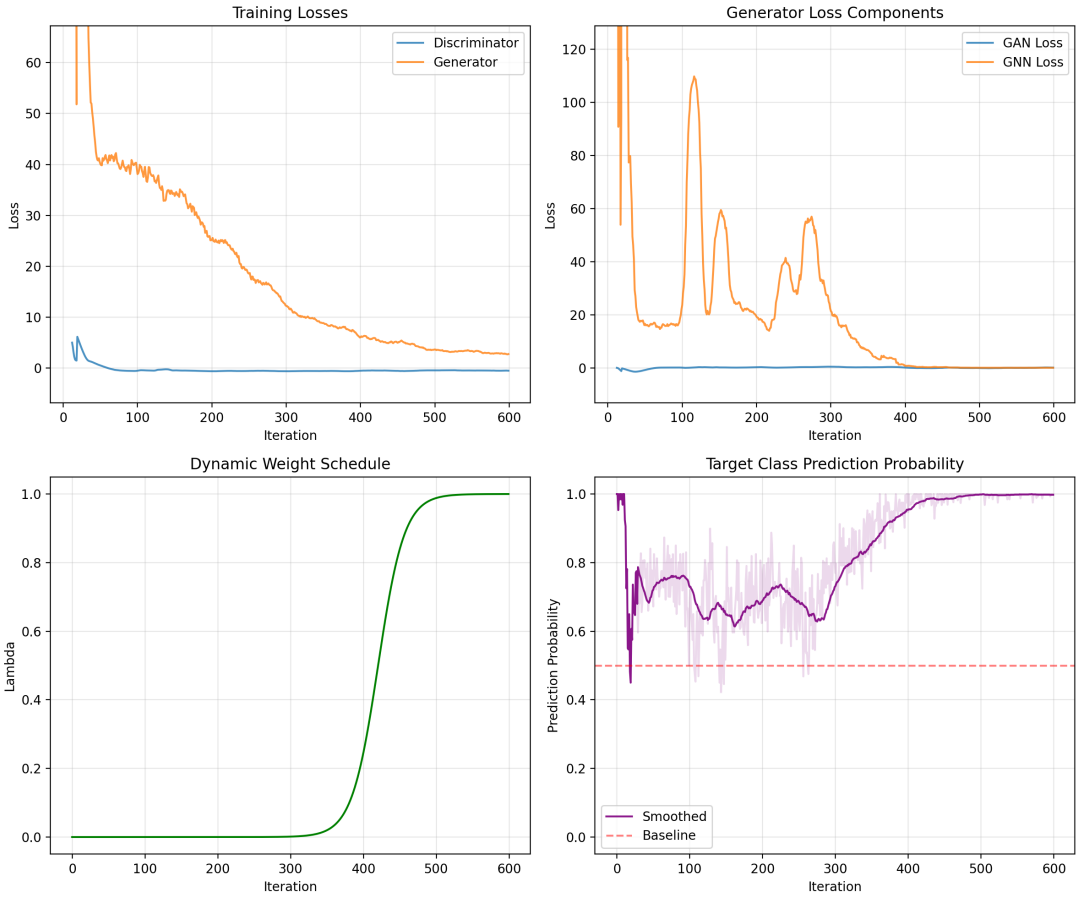


Figure 6: Training curves for GIN-Graph on MUTAG 1-2-GNN class 1, showing generator loss, discriminator loss, and GNN guidance loss over 300 epochs. The dynamic weighting schedule shifts emphasis from adversarial realism (early epochs) to class-specific GNN guidance (later epochs).

larger, indicating that the generators preserve local connectivity more successfully than the global population shape.

Size regularisation helps on PROTEINS, but only partially. On class 0 (Non-Enzyme), generated graphs average 46.6 nodes (1-GNN) and 50.0 nodes (1-2-GNN) versus a real mean of 48.7. On class 1 (Enzyme), the 1-GNN matches the real mean size closely (24.0 vs. 23.7), whereas the 1-2-GNN remains oversized (29.5 vs. 23.7). On MUTAG ($N = 28$), all generators still produce oversized graphs: the generated means lie between 23 and 27 nodes, while the real means lie between 14 and 20. This reflects the fixed generator size and the tendency of the Gumbel-Softmax sampling to keep most nodes active.

Table 8 makes the node-type composition evidence explicit and reveals a mixed rather than uniformly positive picture. On MUTAG class 1 (Non-Mutagen), the 1-2-GNN is the clearest success case: it generates C/O-dominated graphs with 80.0% C and 17.1% O, much closer to the real class distribution than the 1-GNN, which introduces atypical iodine (7.7%) and fluorine (2.0%). On MUTAG class 0 (Mutagen), however, the 1-2-GNN over-emphasises halogens (26.7% F, 11.0% Br, 3.6% I) and underproduces N, whereas the 1-GNN stays closer to the real C/N/O proportions. On PROTEINS class 0, the 1-2-GNN closely matches the real secondary-structure distribution (H: 50.9% vs. 50.4%, S: 47.1% vs. 46.5%, C: 2.0% vs. 3.1%). On PROTEINS class 1, by contrast, both models overproduce sheets and underproduce helix/coil, with the 1-2-GNN class 1 generator being the least faithful (88.9% S vs. 56.2% real).

Table 7: Structural comparison: 500 generated vs. real graphs per configuration. $D_{\mathrm{KS}}^{\mathrm{deg}}$ and $D_{\mathrm{KS}}^{\mathrm{size}}$ denote Kolmogorov–Smirnov statistics on degree and graph-size distributions, respectively. $\bar{d}$ = mean degree, $\bar{n}$ = mean graph size.

Configuration	$D_{\mathrm{KS}}^{\mathrm{deg}} \downarrow$	$D_{\mathrm{KS}}^{\mathrm{size}} \downarrow$	$\bar{d}_{\mathrm{real}}$ / $\bar{d}_{\mathrm{gen}}$	$\bar{n}_{\mathrm{real}}$ / $\bar{n}_{\mathrm{gen}}$
MUTAG				
1-GNN class 0	0.120	0.891	2.09 / 2.06	13.9 / 23.8
1-GNN class 1	0.094	0.893	2.24 / 2.23	19.8 / 27.3
1-2-GNN class 0	0.125	0.881	2.09 / 2.05	13.9 / 23.1
1-2-GNN class 1	0.155	0.594	2.24 / 2.23	19.8 / 24.3
PROTEINS				
1-GNN class 0	0.149	0.660	3.80 / 3.81	48.7 / 46.6
1-GNN class 1	0.213	0.537	3.64 / 3.73	23.7 / 24.0
1-2-GNN class 0	0.178	0.704	3.80 / 3.81	48.7 / 50.0
1-2-GNN class 1	0.165	0.738	3.64 / 3.64	23.7 / 29.5

5.6.2 Cross-Model Classification

The most revealing analysis classifies each set of 500 generated graphs with *both* k-GNN classifiers, testing whether explanations produced by one model are recognisable by the other. Table 9 reports target class agreement rates, and Figure 7 visualises the full classification breakdown.

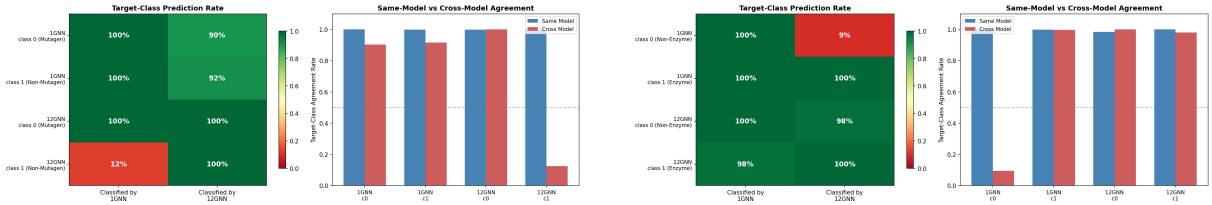
On MUTAG, three of four configurations show high cross-model agreement: 1-GNN class 0 at 90.4%, 1-GNN class 1 at 91.6%, and 1-2-GNN class 0 at 100%. The exception is striking: the 1-2-GNN's Non-Mutagen graphs (class 1) achieve 100% same-model agreement but

Table 8: Population-level node-type composition (%) for the 500-graph comparison. Real distributions are repeated within each class to make within-class comparisons direct.

Dataset / class	Model	Real	Generated
MUTAG c0	1-GNN	C 64.2, N 13.0, O 18.8, F 0.9, I 0.0, Cl 3.0, Br 0.2	C 57.2, N 10.9, O 17.0, F 6.2, I 0.0, Cl 8.6, Br 0.0
MUTAG c0	1-2-GNN	C 64.2, N 13.0, O 18.8, F 0.9, I 0.0, Cl 3.0, Br 0.2	C 45.8, N 0.1, O 12.8, F 26.7, I 3.6, Cl 0.0, Br 11.0
MUTAG c1	1-GNN	C 73.2, N 9.3, O 17.1, F 0.1, I 0.0, Cl 0.2, Br 0.0	C 63.4, N 7.6, O 19.3, F 2.0, I 7.7, Cl 0.0, Br 0.0
MUTAG c1	1-2-GNN	C 73.2, N 9.3, O 17.1, F 0.1, I 0.0, Cl 0.2, Br 0.0	C 80.0, N 2.9, O 17.1, F 0.0, I 0.0, Cl 0.0, Br 0.0
PROTEINS c0	1-GNN	H 50.4, S 46.5, C 3.1	H 54.3, S 45.7, C 0.0
PROTEINS c0	1-2-GNN	H 50.4, S 46.5, C 3.1	H 50.9, S 47.1, C 2.0
PROTEINS c1	1-GNN	H 40.7, S 56.2, C 3.1	H 24.5, S 75.5, C 0.0
PROTEINS c1	1-2-GNN	H 40.7, S 56.2, C 3.1	H 11.0, S 88.9, C 0.0

only 12.4% cross-model agreement—the 1-GNN classifies 87.6% of them as *Mutagen*. A cautious interpretation is that the 1-2-GNN is using pairwise co-occurrence patterns within the carbon scaffold, or relative placement of small N/O motifs, rather than the node-level cues that dominate the 1-GNN prototypes. We cannot identify the exact pair features from these experiments alone, but the flip strongly suggests that the higher-order model's Non-Mutagen boundary depends on structural relationships the 1-GNN does not encode.

On PROTEINS, all four generators succeed at their target class (same-model agreement $\geq$ 98.4%), enabling a clean cross-model comparison. The most revealing asymmetry is on class 0 (Non-Enzyme): 1-GNN-generated Non-Enzyme graphs achieve 100% same-model agreement but only 9.4% cross-model agreement—the 1-2-GNN classifies 90.6% of them as Enzyme. In contrast, 1-2-GNN-generated Non-Enzyme graphs achieve 98.4% same-model agreement *and* 100% cross-model agreement—the 1-GNN also accepts them as Non-Enzyme. On class 1 (Enzyme), by contrast, both models largely agree (98–100% cross-model), suggesting that some class-defining protein patterns are visible to both architectures. Across the two datasets, the asymmetry is therefore class-dependent rather than uniform: transfer breaks down most strongly when one model has collapsed onto a narrower, model-specific prototype family, and it remains high where the class structure seems easier or more shared across architectures.



(a) MUTAG (b) PROTEINS

Figure 7: Cross-model classification of generated graphs. Rows correspond to generator/class combinations and columns to the classifier used for evaluation. Large same-model bars together with sharply lower cross-model bars identify model-specific explanation families rather than universally shared prototypes.

Table 9: Cross-model classification of generated graphs (500 per class). Same = target class agreement by the generating model's classifier; Cross = agreement by the other model's classifier.

Generated by	Target class	Same	Cross
MUTAG			
1-GNN	Mutagen (c0)	100%	90.4%
1-GNN	Non-Mutagen (c1)	99.8%	91.6%
1-2-GNN	Mutagen (c0)	99.8%	100%
1-2-GNN	Non-Mutagen (c1)	100%	12.4%
PROTEINS			
1-GNN	Non-Enzyme (c0)	100%	9.4%
1-GNN	Enzyme (c1)	99.8%	99.6%
1-2-GNN	Non-Enzyme (c0)	98.4%	100%
1-2-GNN	Enzyme (c1)	100%	98.0%

5.6.3 Embedding Space Structure

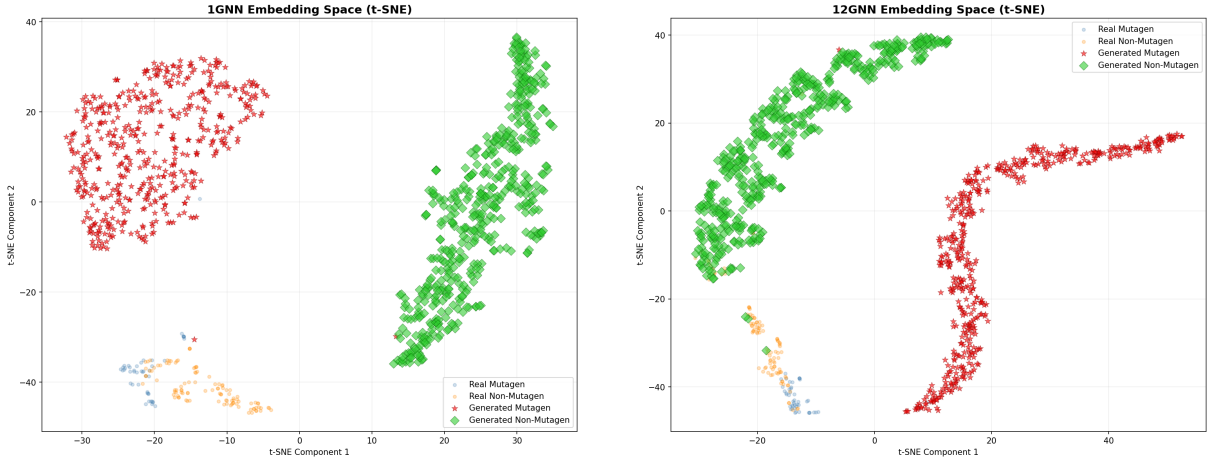
Figures 8 and 9 show t-SNE projections of real and generated graph embeddings in each model's learned representation space. To complement the visualisations, Table 10 reports the displacement of the generated class centroid from the real class centroid, normalised by the real between-class distance *within the same model's embedding space*.

On MUTAG (Figure 8), the generated graphs are cleanly separated by class, but they do not simply fill in the real-data clouds. Instead, each model produces two large generated manifolds that sit away from the much smaller real clusters. The same-model agreement is therefore not coming from faithful sampling of the empirical distribution, but from the generator finding class-consistent regions of embedding space. The normalized drift values support the class-specific picture: the 1-2-GNN drifts much less on Non-Mutagen than on Mutagen (3.93 vs. 17.02), matching its cleaner class-1 population statistics and its much less faithful halogen-heavy class-0 family.

The effect is even stronger on PROTEINS (Figure 9). In the 1-GNN space, the generated Non-Enzyme and Enzyme graphs occupy detached arcs and islands that only partially overlap the real distribution. In the 1-2-GNN space, the generated graphs collapse into several very tight disconnected prototype islands. The centroid-drift statistic should be read carefully here: it measures the location of the generated cloud mean, not its spread. This matters on PROTEINS 1-GNN class 1, where the drift is small (0.38) even though per-graph cosine similarity remains low, implying a dispersed cloud centered near the real class rather than faithful pointwise reproduction. Taken together, the t-SNE plots and drift values support a cautious interpretation of the population-level results: the generators preserve class-discriminative structure well enough to achieve high same-model agreement, but they do not reproduce the full geometry of the real dataset. The generated graphs are best interpreted as model-specific prototypes rather than faithful random samples.

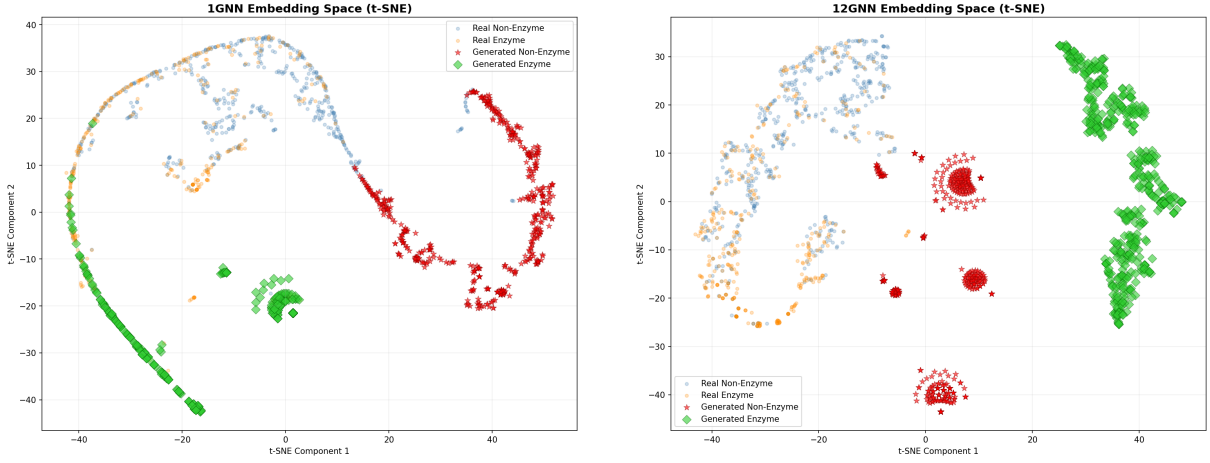
Table 10: Normalized embedding centroid drift, defined within each model's embedding space as $\|\mu_{\text{real},c} - \mu_{\text{gen},c}\| / \|\mu_{\text{real},0} - \mu_{\text{real},1}\|$. Smaller values indicate that the generated class mean lies closer to the real class mean. These values are comparable within a model, but not across models.

Configuration	Class 0 drift	Class 1 drift
MUTAG 1-GNN	5.99	10.19
MUTAG 1-2-GNN	17.02	3.93
PROTEINS 1-GNN	3.51	0.38
PROTEINS 1-2-GNN	1.18	7.02



(a) 1-GNN embedding space (b) 1-2-GNN embedding space

Figure 8: t-SNE projections of MUTAG real (small dots) and generated (large markers) graph embeddings. The generated Mutagen and Non-Mutagen sets form large detached manifolds rather than filling the small real-data clusters, consistent with prototype generation rather than faithful sampling.



(a) 1-GNN embedding space (b) 1-2-GNN embedding space

Figure 9: t-SNE projections of PROTEINS real (small dots) and generated (large markers) graph embeddings. The generated graphs form detached arcs and tight islands, especially in the 1-2-GNN space, showing that class-consistent generation does not imply full reproduction of the real embedding geometry.

6 Discussion

6.1 Different Architectures, Different Representations

Three claims are clearly supported by the evidence. First, the 1-GNN and 1-2-GNN learn different prototype families even when classifier accuracy is comparable. MUTAG is the clearest case: both models reach 89.5% best accuracy, yet one produces N/O-rich mutagenic prototypes while the other produces halogen-heavy ones. Second, these differences are not superficial figure-reading artefacts; the cross-model classification asymmetries in Section 5.6 show that some generated classes transfer poorly across architectures. Third, the generated populations preserve class-discriminative structure without faithfully reproducing the empirical data cloud, which is why the outputs are best interpreted as architecture-specific prototypes rather than samples from the real distribution.

The evidence is mixed on absolute explanation quality. On PROTEINS, the 1-GNN is strongest on Non-Enzyme by the original validation score, while the 1-2-GNN is strongest on Enzyme validity and gives the closest population-level Non-Enzyme node-type match. On MUTAG, the 1-2-GNN is clearly better on Non-Mutagen population composition but clearly worse on Mutagen composition. The most defensible reading is therefore case-specific: higher-order message passing changes what is learned and yields selected fidelity gains, but those gains depend on the class and dataset.

What these experiments do *not* prove is that the 1-2-GNN is universally better or more interpretable. The single-seed design, lack of baselines, and model-specific embedding spaces rule out a blanket superiority claim. The cross-model asymmetries are still important, but they show *difference* more clearly than they show *dominance*. Where we do infer a role for pairwise structure, it should be read as a plausible explanation of the observed patterns rather than as a fully isolated causal mechanism.

6.2 Qualitative Differences in Generated Explanations

Because the percentages below are computed on validation-ranked top explanations, they should be read as representative high-scoring prototypes rather than unbiased population frequencies. Table 8 provides the population-level view and generally confirms the same directional differences with less extreme values.

On MUTAG class 0 (Mutagen), the top-ranked 1-GNN explanations are dominated by carbon, nitrogen, and oxygen (42.9% C, 13.9% N, 13.2% O), whereas the top-ranked 1-2-GNN explanations are much more halogen-heavy (31.1% F, 1.1% Br) and nearly eliminate nitrogen (0.4% N). This difference is consistent with the population-level comparison, where the 1-2-GNN class-0 generator overproduces fluorine and bromine relative to the real class far more strongly than the 1-GNN. Both explanation families remain recognisably mutagenic, but they correspond to different mutagenic signature families rather than to a shared prototype.

On MUTAG class 1 (Non-Mutagen), both models converge on carbon-dominated backbones, but the saved top-ranked explanations still differ sharply in composition. The 1-GNN top set is split between carbon and oxygen (38.6% C, 38.2% O) and introduces substantial iodine (17.9%), whereas the 1-2-GNN top set is more conventional and concentrated on carbon, oxygen, and nitrogen (78.6% C, 14.3% O, 7.1% N). This is the

cleaner success case for the 1-2-GNN: the same directional improvement appears in the population table and in the lower normalized class-1 drift reported in Table 10.

On PROTEINS, both models generate secondary-structure distributions built mainly from helix and sheet nodes, but they differ sharply by class. The 1-2-GNN class 0 generator is the closest match to the real Non-Enzyme distribution, whereas class 1 is much less faithful and becomes strongly sheet-heavy. The 1-GNN produces high-quality Non-Enzyme explanations but struggles with Enzyme validity (37%), while the 1-2-GNN achieves high validity on both classes (100% and 94%) but with lower overall fidelity on class 1 ($s = 0.151$). The resulting picture is not "1-2-GNN is better", but that the two architectures focus on different structural aspects of protein graphs.

6.3 Local Structure vs. Population Fidelity

The generated-dataset comparison reveals an important distinction between *local structural realism* and *population-level fidelity*. Across both datasets, the generators preserve degree distributions much better than graph-size distributions. This means that the local connectivity pattern of a typical generated graph often looks plausible even when the overall population of generated graphs is systematically shifted toward larger or more regular structures.

This pattern is clearest on MUTAG, where all four generators remain oversized despite matching average degree closely. The strongest positive case is 1-2-GNN on class 1 (Non-Mutagen): its node-type proportions and graph sizes are closer to the real class than the corresponding 1-GNN generator. However, this advantage is not universal. On MUTAG class 0, the 1-2-GNN overproduces halogens and diverges more strongly from the real atom distribution than the 1-GNN. On PROTEINS, 1-2-GNN class 0 is the cleanest structural match, while 1-2-GNN class 1 is substantially less faithful than its class-0 counterpart. The most defensible interpretation is therefore not that 1-2-GNN is uniformly "better", but that the two architectures produce different prototype families and that the higher-order model yields the clearest fidelity gain only in selected cases.

6.4 The Granularity Problem

Most generated explanations have granularity $\kappa \approx 0$, meaning they remain close to full-graph prototypes rather than small substructures. This is a structural limitation of the GIN-Graph approach: the generator produces graphs of a fixed maximum size N, and the Gumbel-Softmax sampling tends to activate many nodes. For MUTAG ($N = 28$, average graph $\sim$18 nodes), the generated populations still average 23–27 active nodes. For PROTEINS ($N = 50$, average graph $\sim$26 nodes), the class 0 generators remain near the size cap (46.6 nodes for 1-GNN and 50.0 for 1-2-GNN), while the class 1 generators are somewhat smaller (24.0 and 29.5 nodes) but still do not behave like sparse subgraph explanations.

Model-level explanations are inherently coarse-grained: they answer "what does a typical class member look like?" rather than "which substructure drives the prediction?" This is a property of the method, not a flaw—the two questions require different explanation approaches.

7 Limitations

Several limitations qualify our findings:

Single seed, no error bars. All experiments use a single data split (seed 42) without cross-validation. With only 38 test graphs on MUTAG, the identical 89.5% best accuracy for both models could be coincidence—a difference of one misclassified graph changes accuracy by 2.6 percentage points. The generated-sample counts are also modest: for example, MUTAG 1-GNN class 0 has 36% validity from $n = 50$, which corresponds to an approximate Wilson 95% interval of 24–50%, while MUTAG 1-2-GNN class 1 has 82% validity from $n = 100$, corresponding to roughly 73–88%. These intervals capture only sampling uncertainty within a run; seed-to-seed and split-to-split variation remain unmeasured. This is why we treat the qualitative divergence in generated explanations as stronger evidence than small within-table percentage gaps.

No cross-validation. Best test accuracy is reported as the maximum over epochs (early stopping by oracle), which overestimates generalisation performance. A more rigorous evaluation would use a held-out validation set for model selection.

Classifier–generator coupling. GIN-Graph explanations are shaped by how the classifier's internal representations interact with the generator's optimisation landscape. The 1-2-GNN class 0 (Non-Enzyme) generator on PROTEINS achieves moderate prediction probability ($p = 0.564$), lower than the 1-GNN's near-perfect $p = 0.986$ on the same class. We attribute this gap to an *embedding dimensionality mismatch*: the 1-2-GNN classifier operates on a concatenated embedding $\mathbf{h}_G = [\mathbf{h}_G^{(1)} \| \mathbf{h}_G^{(2)}] \in \mathbb{R}^{128}$, double the 1-GNN's $\mathbb{R}^{64}$, while the generator uses the same architecture in both cases. The 2-GNN component's response to adjacency changes is mediated by pair-level aggregation (Equations 27–28), creating a more complex gradient landscape. However, with graph size and node type regularisation, the generator achieves sufficient prediction probability for meaningful cross-model analysis (98.4% same-model agreement).

The 1-GNN class 1 (Enzyme) generator (37% validity, $s = 0.356$) exhibits a different pattern: $p = 0.919$ indicates the generator can find the target class region, but the generated graphs diverge from the real Enzyme embedding centroid. This is a fidelity problem (the generated graphs are atypical Enzyme structures) rather than a guidance problem (the classifier does recognise them as Enzyme).

Generator–classifier output space mismatch. A fundamental consideration is that GIN-Graph was designed for standard (1-WL) GNNs, and its generator architecture reflects this. The generator outputs an adjacency matrix $\tilde{A}$ and node feature matrix $\tilde{X}$—quantities that a 1-GNN directly consumes. A 1-GNN's decisions are based on node features and local neighbourhoods, which the generator can target directly: "place nitrogen here, connect it to these carbons." The 1-2-GNN, however, makes decisions in a pairwise feature space derived from all $\binom{n}{2}$ node pairs. The generator has no mechanism to directly control pair-level representations—it can only influence them indirectly through edge and node choices. Despite this, the generator achieves high same-model agreement for all configurations ($\geq 94\%$) after structural regularisation, demonstrating that the indirect optimisation path is viable. However, the consistently lower prediction probability for the 1-2-GNN ($p = 0.564$ vs. 0.986 on PROTEINS class 0) suggests that a higher-order–aware generator—one that directly produces pair-level features—could improve results further.

Evaluation metrics. The validation score $v = (s \cdot p \cdot d)^{1/3}$ can be dominated by a

single low component. The degree score, which measures structural realism via average degree alone, is a coarse proxy that does not capture finer structural properties like motif frequencies or degree distributions. The embedding similarity term s is also computed in each model's *own* embedding space, so raw 1-GNN and 1-2-GNN values are informative within model but not strictly apples-to-apples across models. The normalized centroid-drift statistic in Table 10 has the same limitation and, additionally, captures the generated cloud mean rather than within-cloud spread.

No baselines. We do not compare with model-level XGNN [Yuan et al.(2020)]. We also do not compare with instance-level methods such as GNNExplainer [Ying et al.(2019)] and PGExplainer [Luo et al.(2020)]. These methods target different explanatory granularities and are therefore not direct plug-in substitutes for GIN-Graph, but a side-by-side comparison would still contextualise what is gained and lost by model-level prototype generation.

8 Future Work

Several directions could strengthen this investigation:

- **Higher-order–aware generator.** The most impactful extension would be a generator architecture designed for k-GNN classifiers. Rather than outputting only adjacency and node features, such a generator could directly produce pair-level (or k-set–level) features, or use higher-order message passing internally so that it "thinks" in the same representation space as the classifier. This would disentangle the question of whether the 1-2-GNN learns richer structural patterns from the confound of generator–classifier mismatch identified in our limitations.
- Use cross-validation and multiple random seeds for robust accuracy and explanation quality estimates. The current single-seed results, while consistent across configurations, may not generalise.
- Investigate *why* the 1-GNN and 1-2-GNN learn different class representations—e.g., which specific pairwise patterns drive the 1-2-GNN's more selective Non-Enzyme boundary—using attention or gradient-based attribution on the 2-GNN component.
- Extend to the 1-2-3-GNN architecture, which adds 3-set (triplet) message passing for even higher expressiveness, to test whether the interpretability benefit continues up the k-WL hierarchy.
- Incorporate richer structural evaluation metrics beyond average degree, such as clustering coefficients, motif frequencies, or subgraph counts, to better capture the qualitative differences between generated explanations.
- Apply the framework to larger and more diverse datasets where the expressiveness gap between 1-WL and higher-order tests is more pronounced.

9 Conclusion

We introduced two technical pieces needed to study higher-order interpretability with GIN-Graph: a fully differentiable dense wrapper for hierarchical k-GNNs, and a corrected validation metric that measures actual embedding similarity rather than reusing prediction probability as a proxy. With those in place, the experiments show that 1-

GNN and 1-2-GNN learn different class prototype families rather than a single shared representation.

On MUTAG, equal classifier accuracy masks sharply different explanation families, with the 1-GNN favouring N/O-rich mutagenic motifs and the 1-2-GNN favouring halogen-heavy ones. On PROTEINS, the picture is mixed: the 1-GNN is strongest on Non-Enzyme by the original validation score, while the 1-2-GNN produces higher-validity Enzyme explanations and the closest population-level Non-Enzyme node-type match. Cross-model classification makes the divergence concrete, with near-complete failures of transfer on MUTAG 1-2-GNN class 1 and PROTEINS 1-GNN class 0.

The strongest conclusion is therefore limited but useful. Higher-order message passing changes which prototypes are learned and can yield clear case-specific fidelity gains, but these gains are class-dependent and do not amount to universal superiority. The generated graphs are best interpreted as architecture-specific prototypes that expose different internal decision rules and make those differences visible even when test accuracy looks similar.

References

[Borgwardt et al.(2005)] K. M. Borgwardt, C. S. Ong, S. Schönauer, S. V. N. Vishwanathan, A. J. Smola, and H.-P. Kriegel. Protein function prediction via graph kernels. *Bioinformatics*, 21(suppl_1):i47–i56, 2005.

[Cai et al.(1992)] J.-Y. Cai, M. Fürer, and N. Immerman. An optimal lower bound on the number of variables for graph identification. *Combinatorica*, 12(4):389–410, 1992.

[Debnath et al.(1991)] A. K. Debnath, R. L. Lopez de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch. Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. *J. Med. Chem.*, 34(2):786–797, 1991.

[Gulrajani et al.(2017)] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville. Improved training of Wasserstein GANs. In *NeurIPS*, 2017.

[Jang et al.(2017)] E. Jang, S. Gu, and B. Poole. Categorical reparameterization with Gumbel-softmax. In *ICLR*, 2017.

[Luo et al.(2020)] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, and X. Zhang. Parameterized explainer for graph neural network. In *NeurIPS*, 2020.

[Morris et al.(2019)] C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, J. E. Lenssen, G. Rattan, and M. Grohe. Weisfeiler and Leman go neural: Higher-order graph neural networks. In *AAAI*, 2019.

[Sun et al.(2025)] J. Sun, S. Pei, F. Zhang, and N. V. Chawla. GIN-Graph: A generative interpretation network for model-level explanation of graph neural networks. *Neurocomputing*, 2025.

[Ying et al.(2019)] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec. GNNExplainer: Generating explanations for graph neural networks. In *NeurIPS*, 2019.

[Yuan et al.(2020)] H. Yuan, J. Tang, X. Hu, and S. Ji. XGNN: Towards model-level explanations of graph neural networks. In *KDD*, 2020.